Wrapper

Vu Tuyet Trinh
SoICT - HUST

1

Introduction

- Wrappers are components of DI systems that communicate with the data sources
  - sending queries from higher levels in the system to the sources
  - converting replies to a format that can be manipulated by query processor
- Complexity of wrapper depends on nature of data source
  - e.g., source is RDBMS, wrapper's task is to interact with JDBC driver
  - in many cases, wrapper must parse semi-structured data such as HTML pages and tranform it into a set of tuples
  - we focus on this latter case

2

Outline

- Problem definition
- Manual wrapper construction
- Learning-based wrapper construction
- Wrapper learning without schema
  - a.k.a., automatic approaches
- Interactive wrapper construction

3

Data Sources

- Consider data sources that consist of a set of Web pages
- For each source S, assume each Web page displays structured data using a schema $\mathbf{T}_S$ and a format $\mathbf{F}_S$
  - these are common across all pages of the source

(a) *countries.com* (b) *easycalls.com* (c) *greatbooks.com*

4

Data Sources

- These kinds of pages are very common on the Web
  - often created in sites that are powered by database systems
  - user sends a query to the database system
    - e.g., list all countries and calling co in the continent Australia
  - system produces a set of tuples
  - a scripting program creates an HTML page that embeds the tuples , using a schema $\mathbf{T}_s$ and a format $\mathbf{F}_s$
  - the HTML is sent to the user

Wrapper

- A wrapper W extracts structured data from pages of S
- Formally, W is a tuple ($\mathbf{T}_W$, $\mathbf{E}_W$)
  - $\mathbf{T}_W$ is a target schema
    - this needs not be the same as the schema $\mathbf{T}_S$ used on the page, because we may want to extract only a subset of attributes of $\mathbf{T}_S$
  - $\mathbf{E}_W$ is an extraction program that uses format $\mathbf{F}_S$ to extract from each page a data instance conforming to $\mathbf{T}_W$
    - $\mathbf{E}_W$ is typically written in a script language (e.g., Perl) or in some higher-level declarative language that an execution engine can interpret

Example 1

- Consider a wrapper that extracts all attributes from pages of countries.com
  - target schema $\mathbf{T}_W$ is the source schema $\mathbf{T}_S$ = (country, capital, population, continent)
  - extraction program $\mathbf{E}_W$ may be a Perl script that specifies that given a page P from this source
    - return the first fully capitalized string as country
    - return the string immediately following "Capital:" as capital
    - etc.

countries.com

7

Example 2

- Consider a wrapper that extracts only the first two attributes from pages of countries.com
  - target schema $\mathbf{T}_W$ is (country, capital)
  - extraction program $\mathbf{E}_W$ may be a Perl script that specifies that given a page P from this source
    - return the first fully capitalized string as country
    - return the string immediately following "Capital:" as capital

countries.com

8

The Wrapper Construction Problem

- Construct ($\mathbf{T}_W$, $\mathbf{E}_W$) by inspecting the pages of S
  - also called wrapper learning
- Two main variants
  - given schema $\mathbf{T}_W$, construct extraction program $\mathbf{E}_W$
    - e.g., given $\mathbf{T}_W$ = (country, capital), construct $\mathbf{E}_W$ that extracts these two attributes from source countries.com
    - manual/learning/interactive approaches address this problem (see later)
  - no $\mathbf{T}_W$ is given, instead, construct the source schema $\mathbf{T}_S$ and take it to be the target schema $\mathbf{T}_W$, then construct $\mathbf{E}_W$
    - e.g., given pages from source countries.com, learn their schema $\mathbf{T}_S$, then learn $\mathbf{E}_W$ program that extracts the attributes of $\mathbf{T}_S$
    - the automatic approach addresses this problem (see later)

9

Challenges of Wrapper Construction

1. Learning source schema $\mathbf{T}_S$ is very difficult
   - typically view each page of S as a string generated by a grammar G
   - learn G from a set of pages of S, then use G to infer $\mathbf{T}_S$
   - e.g., pages of countries.com may be generated by R = <html>.+?<hr>
(.+?)
Capital: (.+?)
Population: (.+?)
Continent: (.+?)</html> which encodes a regular grammar
   - inferring a grammar from positive examples (i.e., pages of S) is well-known to be difficult
     - regular grammars cannot be correctly identified from positive examples
     - even with both positive and negative examples, there is no efficient algorithm to identify a reasonable grammar (i.e., one that is minimal)

10

Challenges of Wrapper Construction

1. Learning source schema $\mathbf{T}_S$ is very difficult (cont.)
   - current solutions consider only relatively simple regular grammars that encode either flat or nested tuple schemas
   - even learning these simple schemas has proven difficult
   - typically use various heuristics to search a large space of candidate schemas
     - incorrect heuristics often lead to incorrect schemas
     - increasing the complexity of the schema even slightly can lead to an exponential increase in size of search space, resulting in an intractable searching process

Challenges of Wrapper Construction

2. Learning the extraction program $\mathbf{E}_W$ is difficult
   - ideally, $\mathbf{E}_W$ should be Turing complete (e.g., as a Perl script) to have maximal expressive power, but impractical to learn such programs
   - so assume $\mathbf{E}_W$ to be of a far more restricted computational model, then learn only the limited set of parameters of model
     - e.g., learning to extract country and capital from pages of countries.com
     - assume $\mathbf{E}_W$ is specified by a tuple ($\mathbf{s}_1$, $\mathbf{e}_1$, $\mathbf{s}_2$, $\mathbf{e}_2$): $\mathbf{E}_W$ always extract the first string between $\mathbf{s}_1$ and $\mathbf{e}_1$ as country, and the first string between $\mathbf{s}_2$ and $\mathbf{e}_2$ as capital
     - so here learning $\mathbf{E}_W$ reduces to learning the above four parameters
   - even just learning a limited set of parameters has proven quite difficult, for reasons similar to those of learning schema $\mathbf{T}_S$

Challenges of Wrapper Construction

- Coping with myriad exceptions
  - there are often many exceptions in how data is laid out and formatted
    - e.g., (title, author, price) in the normal cases, but attributes (e.g., price) may be missing, attribute order may be reversed (e.g., (author, title, price)), or attribute format may be changed (e.g., price in red font)
  - when inspecting a small number of pages to create wrapper, such exceptions may not be apparent (yet)
  - thus exceptions cause many problems
    - invalidate assumptions on schema/data format, thus producing incorrect wrappers
    - force us to revise source schema $\mathbf{T}_S$ and extraction program $\mathbf{E}_W$, such revisions blow up the search space

Main Solution Approaches

- Manual
  - developer manually creates $\mathbf{T}_W$ and $\mathbf{E}_W$
- Learning
  - developer highlights the attributes of $\mathbf{T}_W$ in a set of Web pages, then applies a learning algorithm to learn $\mathbf{E}_W$
- Automatic
  - automatically learn both $\mathbf{T}_S$ and $\mathbf{E}_W$ from a set of Web pages
- Interactive
  - combines aspects of learning and automatic approaches
  - developer provides feedback to a program until convergence

Outline

- Problem definition
- Manual wrapper construction
- Learning-based wrapper construction
- Wrapper learning without schema
  - a.k.a., automatic approaches
- Interactive wrapper construction

15

Manual Wraper Construction

- Developer examines a set of Web pages
  - manually creates target schema $\mathbf{T}_W$ and extraction program $\mathbf{E}_W$
  - often writes $\mathbf{E}_W$ using a procedural language such as Perl

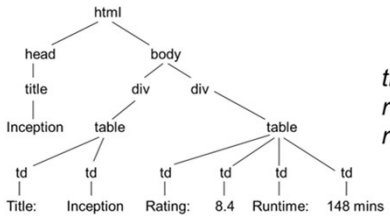

```
#!/usr/bin/perl -w

open(INFILE, $ARGV[0]) or die "can't open file\n";
while ($line = <INFILE>) {
  if ($line =~ m/<B\>(.+?)\<\/B\>\s+?\<I\>(\d+?)\<\/I\>\<BR\>/) {
     print "($1,$2)\n";
  }
}
close(INFILE);
```


16

Manual Wrapper Construction

- There are multiple ways to view a page
  - as a string ➔ can write wrapper as Perl program
  - as a DOM tree ➔ can write wrapper using XPath language



title = /html/body/div[1]/table/td[2]/text()
rating = /html/body/div[2]/table/td[2]/text()
runtime = /html/body/div[2]/table/td[4]/text()

  - as a visual page, consisting of blocks

Manual Wrapper Construction

- Regardless of page model (string, DOM tree, visual, etc.), using a low-level procedural language to write $\mathbf{E}_W$ can be very laborious
- High-level wrapper languages have been proposed
- E.g., HLRT language
  - see the next part on learning
- Using high-level language often result in loss of expressiveness
- But they are often easier to understand, debug, and maintain

Outline

- Problem definition
- Manual wrapper construction
- Learning-based wrapper construction
- Wrapper learning without schema
  - a.k.a., automatic approaches
- Interactive wrapper construction

Learning-Based Wrapper Construction

- Consider more limited wrapper types (compared to the manual approach)
- But can automatically learn these using training examples
- Providing such examples typically involve marking up Web pages
  - can be done by technically naïve users
  - often requires far less work than manually writing wrappers
- We explain learning approaches using two wrapper types
  - HLRT
  - Stalker

HLRT Wrappers

- Use string delimiters to specify how to extract tuples


```
<HTML>                                                  } head
<TITLE>Countries in Australia (Continent)</TITLE>
<BODY>
<B>Countries in Australia (Continent)</B><P>
<B>Australia</B>  <I>61</I><BR>                         } data region
<B>East Timor</B>  <I>670</I><BR>
<B>Papua New Guinea</B>  <I>675</I><BR>
<HR>                                                    } tail
<B>Copyright easycalls.com</B>
</BODY>
</HTML>
```

- To extract (country, code), an HLRT wrapper can
  - chop off the head using <P>, chop off the tail using <HR>
  - extract strings between <B> and </B> in the data region as countries, and between <I> and </I> as codes

HLRT Wrappers

- Thus, HLRT = Head-Left-Right-Tail
- Above wrapper can be represented as tuple (<P>, <HR>, <B>, </B>, <I>, </I>)
- Formally, an HLRT wrapper that extracts n attributes is a tuple of (2n + 2) strings (h, t, l_1, r_1, ..., l_n, r_n)


```
<HTML>                                                  } head
<TITLE>Countries in Australia (Continent)</TITLE>
<BODY>
<B>Countries in Australia (Continent)</B><P>
<B>Australia</B>  <I>61</I><BR>                         } data region
<B>East Timor</B>  <I>670</I><BR>
<B>Papua New Guinea</B>  <I>675</I><BR>
<HR>                                                    } tail
<B>Copyright easycalls.com</B>
</BODY>
</HTML>
```

Learning HLRT Wrappers

- Suppose
  - D wants to extract n attributes $\mathbf{a}_1$, ..., $\mathbf{a}_n$ from source S
  - after examining pages of S, D has established that an HLRT wrapper W = (h, t, $\mathbf{l}_1$, $\mathbf{r}_1$, ..., $\mathbf{l}_n$, $\mathbf{r}_n$) will do the job
- Our goal: learn h, t, $\mathbf{l}_1$, $\mathbf{r}_1$, ..., $\mathbf{l}_n$, $\mathbf{r}_n$
  - to do this, label a set of pages T = {$\mathbf{p}_1$, ..., $\mathbf{p}_m$}
    - i.e., identifying in p_i the start and end positions of all values of attributes $\mathbf{a}_1$, ..., $\mathbf{a}_n$, typically done using a GUI
  - feed the labeled pages $\mathbf{p}_1$, ..., $\mathbf{p}_m$ into a learning module
  - learning module produces h, t, $\mathbf{l}_1$, $\mathbf{r}_1$, ..., $\mathbf{l}_n$, $\mathbf{r}_n$

Example of a Learning Module for HLRT

- A simple module systematically searches the space of all possible HLRT wrappers

1. Find all possible values for h
   - let $\mathbf{x}_i$ be the string from the beginning of page $\mathbf{p}_i$ (a labeled page) until the first occurrence of the very first attribute $\mathbf{a}_1$
   - then $\mathbf{x}_1$, ..., $\mathbf{x}_m$ contains the correct h
   - thus, take the set of all common substrings of $\mathbf{x}_1$, ..., $\mathbf{x}_m$ to be candidate values for h
2. Find all possible values for t:
   - similar to the case of finding all possible values for h

Example of a Learning Module for HLRT

3. Find all possible values for each $\mathbf{l}_i$
   - e.g., consider $\mathbf{l}_1$, the left delimiter of $\mathbf{a}_1$
   - $\mathbf{l}_1$ must be a common suffix of all strings (in labeled pages) that end right before a marked value of $\mathbf{a}_1$
   - can take the set of all such suffixes to be cand values for $\mathbf{l}_1$
4. Find all possible values for each $\mathbf{r}_i$
   - similar to case of $\mathbf{l}_i$, but consider prefixes instead of suffixes
5. Search in the combined space of the above values
   - combine above cand values to form cand wrappers
   - if a cand wrapper W correctly extracts all values of $\mathbf{a}_1$, …, $\mathbf{a}_n$ from all labeled pages $\mathbf{p}_1$, …, $\mathbf{p}_m$, then return W

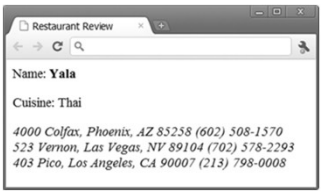
- The notes discuss optimizing the above learning module

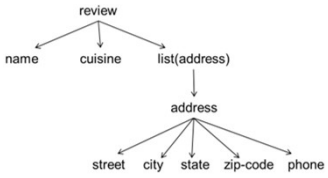
Limitations of HLRT Wrappers

- HLRT wrappers are easy to understand and implement
- But have limited applicability
  - assume a flat tuple schema
  - assume all attributes can be extracted using delimiters
- In practice
  - many sources use more complex schemas, e.g., nested tuples
    - book is modeled as a tuple (title, authors, price), where authors is a list of tuples (first-name, last-name)
  - may not be able to extract using delimiters
    - extracting zip codes from "40 Colfax, Phoenix, AZ 85258"
- Stalker wrappers address these issues

Nested Tuple Schemas

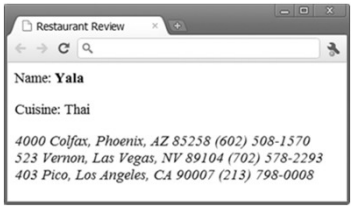
- Stalker wrappers use nested tuple schemas

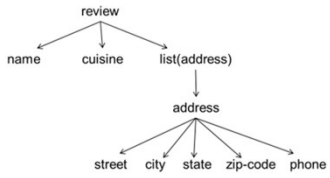




  - here each page is a tuple (name, cuisine, addresses), where addresses is a list of tuples (street, city, state, zipcode, phone)
- Nested tuple schemas are very commonly used in Web pages
  - capture how many people think about the data
  - convenient for visual representation

Nested Tuple Schemas

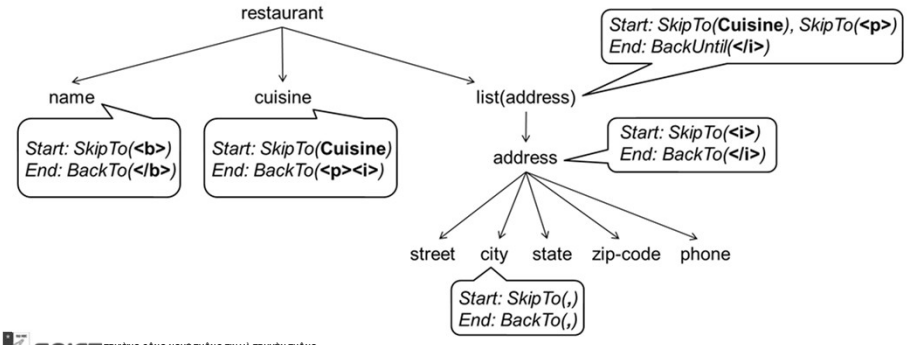




- Definition: let N be the set of all nested tuple schemas
  - the schema displaying data as a single string belongs to N
  - if $\mathbf{T}_1$, ..., $\mathbf{T}_n$ belong to N, then the tuple schema ($\mathbf{T}_1$, ..., $\mathbf{T}_n$) belongs to N
  - if T belongs to N, then the list schema <T> also belongs to N
- A nested tuple schema can be visualized as a tree
  - leaves are strings; internal nodes are tuple or list nodes

The Stalker Wrapper Model

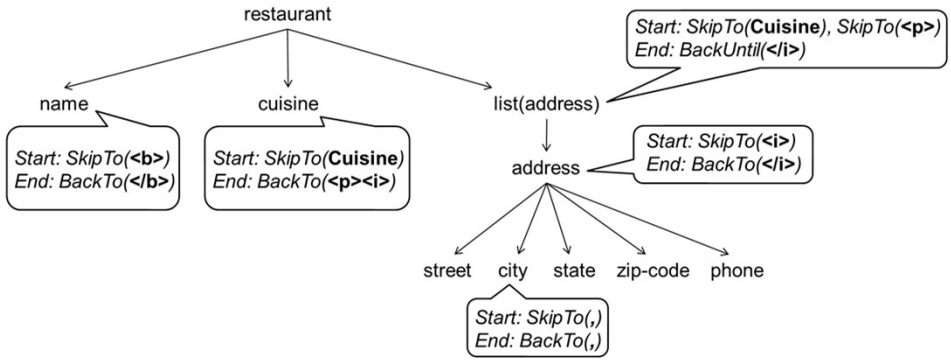
- A Stalker wrapper
  - specifies a nested-tuple schema in form of a tree
  - assigns to each tree node a set of rules that show how to extract values for that node



29

An Example of Executing the Wrapper

```
<p> Name: <b>Yala</b><p>Cuisine: Thai<p>
<i>4000 Colfax, Phoenix, AZ 85258 (602) 508-1570</i><br>
<i>523 Vernon, Las Vegas, NV 89104 (702) 578-2293</i><br>
<i>403 Pico, LA, CA 90007 (213) 798-0008</i>
```



30

Stalker Extraction Rules

- Each rule = context: sequence of commands
- Context: Start, End, etc.
- Sequences of commands: SkipTo(<b>)
  SkipTo(Cuisine:),
  SkipTo(<p>)
- Each command inputs a landmark
  - e.g., <b>, Cuisine:, <p>, or triple (Name Punctuation HTMLTag)
- A landmark = sequence of tokens and wildcards
  - each wildcard refers to a class of tokens
    - e.g., Punctuation, HTMLTag
  - a landmark = a restricted kind of regex

31

Stalker Extraction Rules

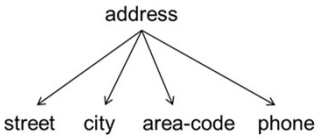
- Each rule = context:sequence of commands
- Executing rule = executing commands sequentially
- Executing command = consuming text until reaching a string that matches the input landmark
- Stalker also considers rules that contain disjunction of sequences of commands
  - Start: either SkipTo(<b>) or SkipTo(<i>)

32

Learning Stalker Wrappers

- Input (of the learner):
  - a nested tuple schema in form of a tree
  - a set of pages where the instances of the tree nodes have been marked up
- Output:
  - use the marked-up pages to learn the rules for the tree nodes
  - for each leaf node: learn a start rule and an end rule
  - for each internal node, e.g., list(address), learn a start rule and an end rule to extract the entire list
- We now illustrate the learning process by considering learning a start rule for a leaf node

Learning a Start Rule for Area Code



E_1: 513 Pico, Venice, Phone: 1-800-555-1515
E_2: 90 Colfax, Palms, Phone: (818) 508-1570
E_3: 523 First St., LA, Phone: 1-888-578-2293
E_4: 403 La Tijera, Watts, Phone: (310) 798-0008

- Use a learning technique called sequential covering
  - 1st iteration: find a rule that covers a subset of training examples
    - e.g., $\mathbf{R}_1$ = SkipTo( ), which covers $\mathbf{E}_2$ and $\mathbf{E}_4$
  - 2nd iteration: find a rule that covers a subset of remaining exams
    - e.g., $\mathbf{R}_7$ = SkipTo(-), which covers all remaining examples
  - and so on, the final rule is a disjunction of all rules found so far
    - e.g., Start: either SkipTo( ) or SkipTo(-)

Learning a Start Rule for Area Code

- Sequential covering can consider a huge number of rules
- Example: consider these rules during the 2nd iteration before selecting the best rule (rule $\mathbf{R}_7$):

Wildcards: *Anything, Numeric, AlphaNumeric, Alphabetic, Capitalized, AllCaps, HTMLTag, nonHTML, Puctuation*

R_7: *SkipTo(***- ***)*
R_8: *SkipTo(Punctuation* *****)*
R_9: *SkipTo(Anything* *****)*
R_{10}: *SkipTo*(**Venice**) *SkipTo*(****)
R_{11}: *SkipTo*(****) *SkipTo*(****)
R_{12}: *SkipTo*(**:**) *SkipTo*(****)
R_{13}: *SkipTo*(**-**) *SkipTo*(****)
R_{14}: *SkipTo*(**,**) *SkipTo*(****)
R_{15}: *SkipTo*(**Phone**) *SkipTo*(****)
R_{16}: *SkipTo*(**1**) *SkipTo*(****)
R_{17}: *SkipTo*(*Numeric*) *SkipTo*(****)
R_{18}: *SkipTo*(*Punctuation*) *SkipTo*(****)
R_{19}: *SkipTo*(*HTMLTag*) *SkipTo*(****)
R_{20}: *SkipTo*(*AlphaNum*) *SkipTo*(****)
R_{21}: *SkipTo*(*Alphabetic*) *SkipTo*(****)
R_{22}: *SkipTo*(*Capitalized*) *SkipTo*(****)
R_{23}: *SkipTo*(*NonHTML*) *SkipTo*(****)
R_{24}: *SkipTo*(*Anything*) *SkipTo*(****)

Discussion

- The wrapper model of Stalker subsumes that of HLRT
  - nested tuple schemas are more general than flat tuple schemas
- Both can be viewed as modeling finite state automata
- Both illustrate how imposing structure on the target schema language makes learning practical
  - structure can be simple as flat tuple schema, or more complex, as nested tuple schemas
  - significantly restrict target language, and transform general learning into a far easier problem of learning a relatively small set of parameters: delimiting strings or extraction rules
- Even with such restricted problem settings, learning is still very difficult: large search space, use of heuristics

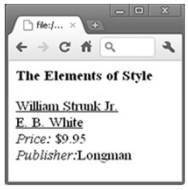
Outline

- Problem definition
- Manual wrapper construction
- Learning-based wrapper construction
- Wrapper learning without schema
  - a.k.a., automatic approaches
- Interactive wrapper construction

37

Wrapper Learning without Schema

- Also called automatic approaches to wrapper learning
  - input a set of Web pages of source S
  - examine similarities and dissimilarities across pages
  - automatically infer schema $\mathbf{T}_S$ of pages and extraction program $\mathbf{E}_W$ that extracts data conforming to $\mathbf{T}_S$



AB+C?D

<HTML><B>#PCDATA</B><P>(<U>#PCDATA</U>
)+
(<I>Price:</I>#PCDATA
)?<I>Publisher:</I>#PCDATA
</HTML>

The Elements of Style	William Strunk Jr.	$9.95	Longman
	E. B. White		
The Snow of Kilimanjaro	Ernest Hemingway		Scribner

38

RoadRunner: A Representative Approach

- Web pages of source S use schema $\mathbf{T}_S$ to display data
- RoadRunner models $\mathbf{T}_S$ as a nested tuple schema
  - allows optionals (e.g., C in AB+C?D)
  - but does not allow disjunctions (would blow up run time)
  - so $\mathbf{T}_S$ here is union-free regular expressions
- Roadrunner models extraction program $\mathbf{E}_W$ as a regex that when evaluated on a Web page will extract attributes of $\mathbf{T}_S$
  - e.g., `<HTML><B>#PCDATA</B><P>(<U>#PCDATA</U><BR>)+ (<I>Price:</I>#PCDATA<BR>)?<I>Publisher:</I>#PCDATA<BR></HTML>`
  - #PCDATA are slots for values, which can't contain HTML tags

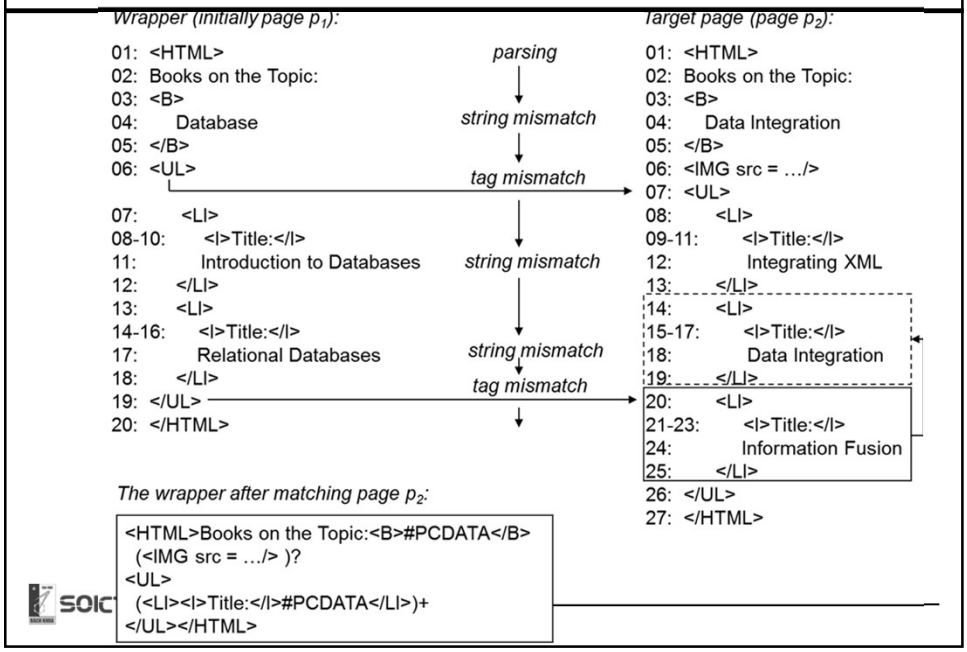
Inferring Schema T_S and Program E_W

- Given set of Web pages P = {$\mathbf{p}_1$, …, $\mathbf{p}_n$}, examine P to infer $\mathbf{E}_W$, then infer $\mathbf{T}_S$ from $\mathbf{E}_W$
- To infer $\mathbf{E}_W$, iterate
  - initializing $\mathbf{E}_W$ to page $\mathbf{p}_1$ (which can be viewed as a regex)
  - generalize $\mathbf{E}_W$ to also match $\mathbf{p}_2$, and so on
  - return an $\mathbf{E}_W$ that has been generalized (minimally) to match all pages in P
- Generalization step is the key, which we discuss next

The Generalization Step

- Assume E_W has been initialized to page p_1
- Now generalize to match page p_2
- Tokenize pages into tokens (string or HTML tag)
- Compare two pages, starting from the first token
- Eventually, will likely to run into a mismatch (of tokens)
  - string mismatch: "Database" vs. "Data Integration"
  - tag mismatch: 2 tags, or 1 tag and 1 string
    - e.g., <UL> vs. <IMG ...>
  - resolving a string mismatch is not too hard, resolving a tag mismatch is far more difficult

41



42

Handling Tag Mismatch

- Due to either an iterator or an optional
  - <UL> vs. <IMG src=.../> is due to an optional image on $\mathbf{p}_2$
  - </UL> on line 19 of $\mathbf{p}_1$ vs. <LI> on line 20 of $\mathbf{p}_2$ is due to an iterator (2 books in $\mathbf{p}_1$ vs. 3 books in $\mathbf{p}_2$)
- When tag mismatch happens
  - try to find if it's due to an iterator
  - if yes, generalize E_W to incorporate iterator
  - otherwise generalize E_W to incorporate optional
  - there is a reason why we look for iterator before looking for optional
    - if we don't do so, everything will be thought of as optional, and be generalized accordingly

Handling Tag Mismatch

- Generalize $\mathbf{E}_W$ to incorporate an optional
  - detect which page includes the optional
    - in the running example, <IMG src=.../> is the optional string
  - generalize $\mathbf{E}_W$ accordingly
    - e.g., introducing the pattern (<IMG src=.../>)?
- Generalize $\mathbf{E}_W$ to incorporate an iterator
  - an iterator repeats a pattern, which we call a square
    - e.g., each book description is a square
  - find the squares, use them to find the lists, then generalize $\mathbf{E}_W$

Handling Tag Mismatch

- Resolving an iterator mismatch often involves recursion
  - while resolving an outer mismatch, may run into an inner mismatch
  - mismatches must be resolved from inside out, recursively

```
<LI>
  Information Fusion
  <B>David Smith</B>
  <B>Jane Lee</B>
</LI>
```

```
<LI>
  Data Integration
  <B>James Madison</B>
</LI>
```


Summary

- To generalize $\mathbf{E}_W$ to match a page p
  - must detect and resolve all mismatches
  - for each mismatch, must decide if it is a string mismatch, iterator mismatch, or optional mismatch
  - for an iterator or optional mismatch, can search on either the side of $\mathbf{E}_W$ (e.g., page $\mathbf{p}_1$) or the side of the target page p
    - e.g., for optional mismatch, the optional can be on either $\mathbf{E}_W$ or p
  - for an iterator or optional mismatch, even when we limit the search to just one side, there are often many square candidates and optional candidates to consider
  - to resolve an iterator mismatch, it may be necessary to recursively resolve many inner mismatches first

Reducing Runtime Complexity

- From summary, it is clear that the search space is vast
  - multiple options at each decision point
  - when dead end, must backtrack to the closest decision point and try another option
  - the generalization algorithm incurs exponential time in the length of the inputs
- RoadRunner uses three heuristics to reduce runtime
  - limits # of options at each decision point, consider only top k
  - does not allow backtracking at certain decision points
  - ignores certain iterator/optional patterns judged to be highly unlikely

Outline

- Problem definition
- Manual wrapper construction
- Learning-based wrapper construction
- Wrapper learning without schema
  - a.k.a., automatic approaches
- Interactive wrapper construction

Motivation

- Limitations of learning and automatic approaches
  - use heuristics to reduce search time in huge space of cands
  - such heuristics are not perfect, so approaches are brittle
    - we have no idea when they produce correct wrappers
  - even with heuristics, still takes way too long to search
- Interactive approaches address these problems
  - start with little or no use input, search until uncertainty arises
  - ask user for feedback, then resume searching
  - repeat until converging to a wrapper that user likes

Motivation

- User feedback can take many forms
  - label new pages, identify correct extraction result, visually create extraction rules, answer questions posed by system, identify page patterns, etc.
- Key challenges
  - decide when to solicit feedback
  - what feedback to solicit
- Will describe three representative systems
  - interactive labeling of pages with Stalker
  - Identifying correct extraction results with Poly
  - Creating extraction rules with Lixto

Interactive Labeling of Pages with Stalker

- Modify Stalker so that it ask user to label pages during the search process (not "before", as discussed so far)
  - asks user to label a page (or a few)
  - uses this page to build an initial wrapper
  - interleaves search with soliciting user feedback until finding a satisfactory wrapper
- How to find which page to ask user to label next?
  - maintain two candidate wrappers
  - find pages on which they disagree
  - ask user to label one of these problematic pages
  - this is a form of active learning called co-testing

51

Detailed Algorithm

1. User labels one or several Web pages

Name:Savory<p>Phone: (608) 263-4567 <p>Fax:(608) 523-4917

2. Learn two wrappers
   - e.g., learning to mark the start of a phone number: we can learn a forward rule as well as a backward rule
     - forward rule $\mathbf{R}_1$: SkipTo(Phone:)
     - backward rule $\mathbf{R}_2$: BackTo(Fax), BackTo(()
3. Apply learned wrappers to find a problematic page
   - apply them to a large set of unlabeled pages
   - if they disagree in extraction results on a page ➔ problematic
4. Ask user to label a problematic page
5. Repeat Steps 2-4 until no more problematic pages

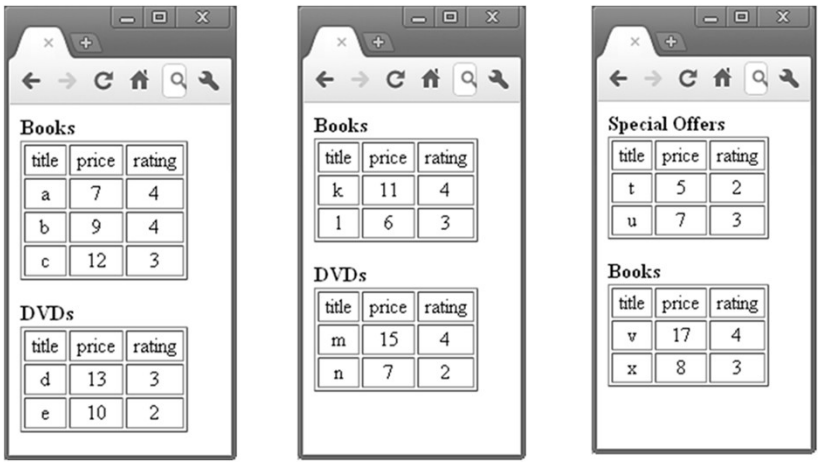
52

Identifying Correct Extraction Results with Poly

- Also uses co-testing, but differs from Stalker
  - maintains multiple cand wrappers instead of just two
  - asks user to identify correct extraction results
  - instead of using string model, uses DOM tree and visual model

1. Initialization
   - assumes multiple tuples per page, assume user wants to extract a subset of these tuples
   - thus, asks user to label a target tuple on a page by highlighting the attributes of the tuple
     - e.g., extracting all tuples (title, price, rating) with rating 4 in Table Books
     - user highlights the first tuple (a, 7, 4) in Table Books

Example

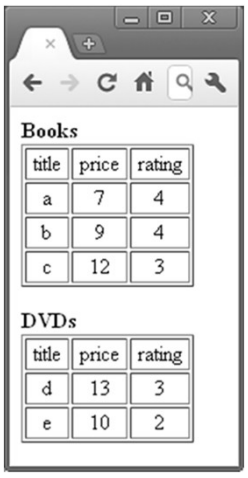


Identifying Correct Extraction Results with Poly

2. Using the labeled tuple to generate multiple wrappers
   - generate multiple wrappers, each extracts from current page a set of tuples that contain the highlighted tuple
   - example wrappers
     - extracts all book and DVD tuples, just book tuples, book and DVD tuples with rating 4, just book tuples with rating 4, the first tuple of all tables, just the first tuple of the first table
     - all of these wrappers extract the highlighted tuple (a, 7, 4)
3. Soliciting the correct extraction result
   - shows user extraction result produced by cand wrappers on the page, and asks user to identify the correct result
   - remove all cand wrappers that do not produce that result

Identifying Correct Extraction Results with Poly

3. Soliciting the correct extraction result (cont.). Example
   - user wants all books with rating 4, so identifies the set {(a, 7, 4), (b, 9, 4)} as correct
   - this removes several wrappers, still leaves those that extract all book and DVD tuples with rating 4, book tuples with rating 4, and all tuples with rating 4 from the first table
   - all of the remaining wrappers still produce correct results on the highlighted page; this page is no longer useful



Identifying Correct Extraction Results with Poly

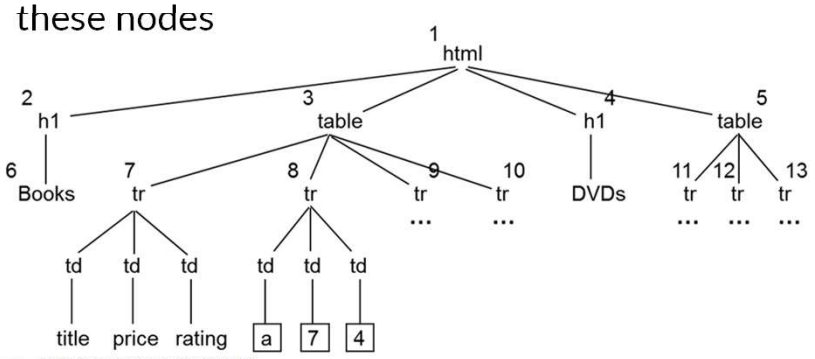
4. Evaluating the remaining wrappers on verification pages
   - applies all remaining wrappers to a large set of unlabeled pages to see if wrappers disagree
     - e.g., "extract all book and DVD tuples with rating 4" and "extract all book tuples with rating 4" disagree on the first page here
     - "extract book tuples with rating 4" and "extract all tuples with rating 4 in the first table" disagree on the second page
   - if finds a disagreement on a page q, repeat Steps 3-4: asks user to select the correct result on q, etc.

5. Return all cand wrappers when they no longer disagree on unlabeled pages

57

Generating the Wrappers in Poly

- Convert page into a DOM tree
- Identifies nodes that map to highlighted attributes
- Create XPath-like expressions from root to these nodes



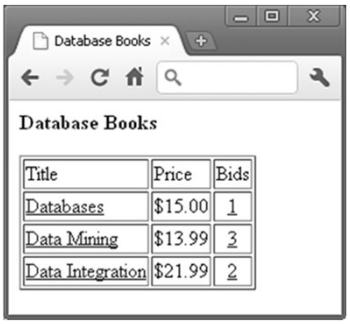
58

Creating Extraction Rules with Lixto

- Lixto vs. Poly and Stalker
  - user visually create extraction rules using highlighting and dialog boxes
    - instead of labeling pages or identifying extraction results
  - encodes extraction rules internally using a Datalog-like language, defined over DOM tree and string models of pages
- Creating the extraction rules visually
  - Web page lists books being auctioned
  - user can create 4 rules
    - Rule 1 extracts books themselves
    - Rules 2-4 extract title/price/# of bids of each book, respectively

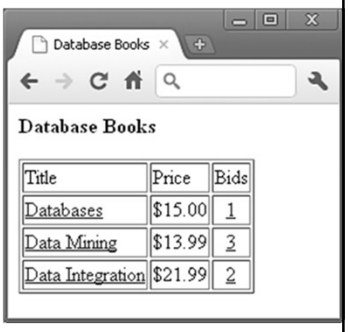
Creating the Extraction Rules with Lixto

- To create Rule 1, which extracts books
  - user highlights a book tuple
    - e.g., the first one “Databases”
  - Lixto maps this tuple to corresponding subtree of the DOM tree of page, extrapolates to create Rule 1, shows result of Rule 1 on the page
  - user accepts Rule 1
    - can also refine the rule



Creating the Extraction Rules with Lixto

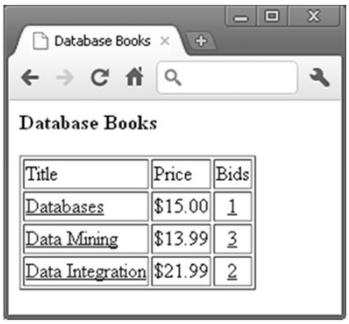
- To create Rule 2, which extracts titles
  - user specifies that this rule will extract from the book instances identified by Rule 1
  - user highlights a title
  - Lixto uses this to create a rule and shows user all extraction results of this rule
  - user realizes this rule is too general (e.g., extracting both titles and bids), so user refines the rule using dialog boxes



61

Creating the Extraction Rules with Lixto

- What can user do?
  - highlighting a tuple or a value
  - using dialog boxes to restrict or relax a rule
  - write regular expressions
  - refers to real-world concepts defined by Lixto



62

Representing the Extraction Rules

- Using a Datalog-like internal language
- User is not aware of this language
- Example of the four rules discussed so far

R_1: *book(S,X)* ← *page(S), subelem(S,.table,X)*
R_2: *title(S,X)* ← *book(-,S), subelem(S,(*.td.*.content,[(href,,substr)]),X), notbefore(S,X,.td,100)*
R_3: *price(S,X)* ← *book(-,S), subelem(S,(*.td,[(elementtext,\var[Y].*,regvar)]),X), isCurrency(Y)*
R_4: *bids(S,X)* ← *book(-,S), subelem(S,*.td,X), before(S,X,.td,0,30,Y,-), price(-,Y)*

Summary

- Critical problem in data integration
  - where many sources produce HTML data
- Huge amount of literature
- Remains very difficult
- Common approaches
  - Manual wrapper construction
  - Learning-based wrapper construction
  - Wrapper learning without schema
    - a.k.a., automatic approaches
  - Interactive wrapper construction